Kubernetes

Complete Guide – From Scratch

Architecture • Pods • ReplicaSets • Deployments

Services • ConfigMaps • Secrets • Namespaces

Ingress • Storage • Autoscaling • Microservices on K8s

Clear Definitions • YAML Manifests • kubectl Commands • Real-World Examples

Table of Contents

CHAPTER 1

Kubernetes Architecture & Core Concepts

Clusters, Control Plane, Worker Nodes, Objects & kubectl

1.1 What is Kubernetes?

Kubernetes (K8s)

Kubernetes is an open-source container orchestration platform originally developed at Google and donated to the CNCF in 2014. It automates the deployment, scaling, and management of containerised applications. Kubernetes answers the question: 'How do I run hundreds of Docker containers reliably across many servers, ensure they stay running, scale them up and down, and update them without downtime?'

- **Self-healing** – Restarts failed containers, replaces and reschedules them when nodes die.
- **Auto-scaling** – Scales pods up/down based on CPU, memory, or custom metrics.
- **Rolling updates** – Updates app versions with zero downtime; rolls back on failure.
- **Service discovery** – Built-in DNS; services find each other by name automatically.
- **Load balancing** – Distributes traffic across healthy pod instances automatically.
- **Secret management** – Stores and manages sensitive information (passwords, tokens).
- **Storage orchestration** – Mounts storage from local, cloud, or network providers.
- **Declarative configuration** – Describe desired state; K8s reconciles the actual state.

1.2 Cluster Architecture

Component	Node	Role
kube-apiserver	Control Plane	Front-end for K8s API; all communication goes through it
etcd	Control Plane	Distributed key-value store; source of truth for cluster state
kube-scheduler	Control Plane	Watches for new pods; assigns them to nodes based on resources
kube-controller-manager	Control Plane	Runs controllers: Node, Replication, Endpoint, ServiceAccount
cloud-controller-manager	Control Plane	Manages cloud-provider-specific resources (LB, volumes, nodes)
kubelet	Worker Node	Agent on each node; ensures containers are running in pods

Component	Node	Role
kube-proxy	Worker Node	Maintains network rules; enables Services to route traffic to pods
Container Runtime	Worker Node	Runs containers (containerd, CRI-O, Docker)
CoreDNS	Control Plane	Cluster DNS; resolves service names to ClusterIP addresses

Cluster architecture diagram

```
# A Kubernetes cluster:

+---------------------------------- CLUSTER ----------------------------------+
| |
| +--- CONTROL PLANE ---+ +--- WORKER NODE 1 ---+ |
| | kube-apiserver | | kubelet | |
| | etcd | <-----> | kube-proxy | (runs Pod A, Pod B) |
| | scheduler | | container runtime | |
| | controller-manager | +---------------------+ |
| +---------------------+ |
| +--- WORKER NODE 2 ---+ |
| | kubelet | |
| | kube-proxy | (runs Pod C, Pod D) |
| | container runtime | |
| +---------------------+ |
| |
| Developer --> kubectl --> kube-apiserver --> etcd, scheduler, controllers |
+-------------------------------------------------------------------------------+
```

1.3 Core Kubernetes Objects

Object	Purpose	Manages
Pod	Smallest deployable unit; 1+ containers that share network/storage	Containers
ReplicaSet	Ensures N copies of a pod are running at all times	Pods
Deployment	Declarative updates for Pods/ReplicaSets; rolling updates/rollback	ReplicaSets
StatefulSet	Like Deployment but for stateful apps with stable identity/storage	Pods (ordered)
DaemonSet	Runs one pod copy on every node (logging, monitoring agents)	Pods per node
Job	Runs pods to completion; for batch processing	Pods (finite)
CronJob	Runs Jobs on a schedule (cron-style)	Jobs
Service	Stable network endpoint; load balances across pod replicas	Pod endpoints

Object	Purpose	Manages
Ingress	HTTP/HTTPS routing from outside the cluster to Services	Services
ConfigMap	Stores non-sensitive configuration data as key-value pairs	Config
Secret	Stores sensitive data (passwords, tokens) base64-encoded	Sensitive config
Namespace	Virtual cluster within a cluster; scope for resource isolation	Resource scoping
PersistentVolume	Cluster-wide storage resource provisioned by admin	Storage
PersistentVolumeClaim	Request for storage by a pod; binds to a PV	Storage request
ServiceAccount	Identity for pods to authenticate to the K8s API	Pod identity
NetworkPolicy	Firewall rules for pod-to-pod and external communication	Network rules
HorizontalPodAutoscaler	Scales pod replicas based on metrics	Deployment/RS scale

1.4 Installing Kubernetes – Local Development

Local Kubernetes setup

```
# ■■ minikube (easiest for local dev) ■■■■■■■■■■■■■■■■■■■■■■■■
# Install minikube: https://minikube.sigs.k8s.io
minikube start # start single-node cluster
minikube start --cpus 4 --memory 8g # with more resources
minikube start --nodes 3 # multi-node cluster
minikube status # check status
minikube stop # stop cluster
minikube delete # delete cluster
minikube dashboard # open K8s web UI
minikube tunnel # enable LoadBalancer services locally

# ■■ kind (Kubernetes IN Docker) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kind create cluster --name my-cluster
kind get clusters
kind delete cluster --name my-cluster

# ■■ kubectl installation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Linux:
curl -LO "https://dl.k8s.io/release/$(curl -Ls
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# macOS:
brew install kubectl

# Verify
kubectl version --client
kubectl cluster-info
```

1.5 kubectl – Essential Commands

Essential kubectl commands

```
# ■■ Context & Cluster Management ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl config get-contexts # list all contexts
kubectl config current-context # show active context
kubectl config use-context prod-cluster

# ■■ Applying & Deleting Manifests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl apply -f pod.yaml # create or update from file
kubectl apply -f ./k8s/ # apply all YAMLs in directory
kubectl apply -f https://raw.github.com/.../manifest.yaml
kubectl delete -f pod.yaml # delete objects defined in file

# ■■ Getting Resources ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl get pods # list pods in current namespace
kubectl get pods -n kube-system # specific namespace
kubectl get pods --all-namespaces # all namespaces
kubectl get pods -o wide # extra info: node, IP
kubectl get pods -o yaml # full YAML output
kubectl get all # pods, services, deployments, etc.
kubectl get nodes # list cluster nodes

# ■■ Describing Resources ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl describe pod my-pod
kubectl describe node worker-1
kubectl describe deployment order-service

# ■■ Logs & Exec ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl logs my-pod # pod logs
kubectl logs -f my-pod # follow logs
kubectl logs my-pod -c sidecar # specific container
kubectl logs -f deployment/order-service
kubectl exec -it my-pod -- bash # interactive shell
kubectl exec my-pod -- ls /app # run command

# ■■ Scaling & Rolling Out ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl scale deployment order-service --replicas=5
kubectl rollout status deployment/order-service
kubectl rollout history deployment/order-service
kubectl rollout undo deployment/order-service

# ■■ Port Forwarding (local dev) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl port-forward pod/my-pod 8080:8080
kubectl port-forward svc/order-service 8080:80

# ■■ Namespace ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl create namespace production
kubectl get pods -n production
kubectl config set-context --current --namespace=production
```

CHAPTER 2

Pods – The Atomic Unit of Kubernetes

Pod Lifecycle, Multi-Container Patterns, Resources, Probes & Scheduling

2.1 What is a Pod?

Pod

A Pod is the smallest and simplest deployable unit in Kubernetes. A Pod represents one or more containers that are tightly coupled and share the same network namespace (same IP, same port space) and storage volumes. Containers within a Pod can communicate via localhost and share mounted volumes. Pods are ephemeral — they are created and destroyed; they do NOT self-heal. That is the job of higher-level objects like ReplicaSets and Deployments.

- Each Pod gets a unique IP address within the cluster.
- Containers in a Pod share localhost — they can talk to each other on localhost:port.
- Pods are mortal — if a node dies, the pods on it are gone. Use Deployments for resilience.
- Rarely create Pods directly — always use Deployments or StatefulSets.
- Pods run on a single node — they cannot span multiple nodes.

2.2 Pod Lifecycle Phases

Phase	Description
Pending	Pod accepted; waiting to be scheduled on a node, or pulling container images
Running	Pod bound to a node; at least one container is running
Succeeded	All containers terminated successfully with exit code 0 (Job pods)
Failed	At least one container terminated with non-zero exit code
Unknown	Pod state cannot be determined (node communication failure)
Terminating	Pod is being deleted; receiving SIGTERM; waiting for gracePeriod

2.3 Pod YAML – Complete Example

Complete Pod YAML with resources, probes, env, volumes

```
# pod.yaml – Single container pod with all key fields
apiVersion: v1
kind: Pod
metadata:
name: order-service-pod
namespace: production
```

```
  labels:
  app: order-service
  version: "2.1"
  tier: backend
  annotations:
  description: "Order service API pod"
  prometheus.io/scrape: "true"
  prometheus.io/port: "8080"
  spec:
  containers:
  - name: order-service
  image: myregistry.io/order-service:v2.1
  imagePullPolicy: Always # Always | IfNotPresent | Never
  ports:
  - containerPort: 8080
  name: http
  protocol: TCP
  env:
  - name: SPRING_PROFILES_ACTIVE
  value: "kubernetes"
  - name: DB_PASSWORD
  valueFrom:
  secretKeyRef:
  name: db-secret
  key: password
  - name: APP_NAME
  valueFrom:
  configMapKeyRef:
  name: app-config
  key: app.name
  resources:
  requests:
  memory: "256Mi"
  cpu: "250m" # 250 millicores = 0.25 CPU core
  limits:
  memory: "512Mi"
  cpu: "500m"
  livenessProbe:
  httpGet:
  path: /actuator/health/liveness
  port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 3
```

```
  readinessProbe:
  httpGet:
  path: /actuator/health/readiness
  port: 8080
  initialDelaySeconds: 20
  periodSeconds: 5
  failureThreshold: 3
  volumeMounts:
  - name: app-config-vol
  mountPath: /app/config
  readOnly: true
  - name: logs
  mountPath: /app/logs
  volumes:
  - name: app-config-vol
  configMap:
  name: order-service-config
  - name: logs
  emptyDir: {}
  restartPolicy: Always # Always | OnFailure | Never
  terminationGracePeriodSeconds: 60
  serviceAccountName: order-service-sa
  imagePullSecrets:
  - name: registry-secret
```

2.4 Multi-Container Pods – Sidecar Pattern

Sidecar Pattern

A sidecar container runs alongside the main application container in the same pod, augmenting or extending its functionality without modifying the main app. Common sidecars: log shippers (Fluentd), service mesh proxies (Envoy/Istio), metric exporters, init scripts, and secret rotators.

Multi-container Pod with sidecar and proxy

```
  # Multi-container pod – main app + logging sidecar + Envoy proxy
  apiVersion: v1
  kind: Pod
  metadata:
  name: order-service-with-sidecar
  spec:
  containers:
  # ■■ Main application container ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  - name: order-service
  image: myregistry/order-service:v2
  ports:
  - containerPort: 8080
  volumeMounts:
```

```
- name: shared-logs
mountPath: /app/logs

# ■■ Sidecar: Fluentd log shipper ■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: log-shipper
image: fluent/fluentd:v1.16
volumeMounts:
- name: shared-logs
mountPath: /app/logs # reads logs from same volume
- name: fluentd-config
mountPath: /fluentd/etc

# ■■ Sidecar: Envoy service mesh proxy ■■■■■■■■■■■■■■■■■■■■
- name: envoy-proxy
image: envoyproxy/envoy:v1.28
ports:
- containerPort: 9901 # admin port
command: ["/usr/local/bin/envoy", "-c", "/etc/envoy/envoy.yaml"]

volumes:
- name: shared-logs # shared between main app and log shipper
emptyDir: {}
- name: fluentd-config
configMap:
name: fluentd-config
```

2.5 Init Containers

Init Container

Init containers run to completion BEFORE the main app containers start. They are used for setup tasks: waiting for a database to be ready, running database migrations, cloning a git repository, or setting up file permissions. If an init container fails, Kubernetes restarts it until it succeeds.

Init containers for startup dependencies

```
spec:
initContainers:

# Init 1: Wait for database to be ready
- name: wait-for-db
image: busybox:1.36
command:
- sh
- -c
- |
until nc -z postgres-service 5432; do
echo "Waiting for PostgreSQL..."; sleep 3;
done
echo "PostgreSQL is ready!"

# Init 2: Run database migrations
```

```
- name: db-migrate
image: myregistry/order-service:v2
command: ["java", "-jar", "app.jar", "--spring.batch.job.enabled=true", "--migrate-only"]
env:
- name: SPRING_DATASOURCE_URL
valueFrom:
secretKeyRef:
name: db-secret
key: url

containers:
- name: order-service
image: myregistry/order-service:v2
# Main app starts only AFTER both init containers succeed
```

2.6 Liveness, Readiness & Startup Probes

Probe	Failure Action	Use For
livenessProbe	Kill & restart the container	Detect deadlock/stuck process; app no longer healthy
readinessProbe	Remove pod from Service endpoints	App not ready to receive traffic (warming up, DB migration)
startupProbe	Kill container if startup too slow	Slow-starting apps; delays liveness probe until started

All three probe types with all configuration options

```
# Probe types:

# 1. HTTP GET – success if status code 200-399
livenessProbe:
httpGet:
path: /actuator/health/liveness
port: 8080
httpHeaders:
- name: Custom-Header
value: probe
initialDelaySeconds: 30 # wait 30s before first check
periodSeconds: 10 # check every 10s
timeoutSeconds: 5 # fail if no response in 5s
failureThreshold: 3 # restart after 3 consecutive failures
successThreshold: 1 # 1 success = healthy

# 2. TCP Socket – success if connection accepted
readinessProbe:
tcpSocket:
port: 5432
initialDelaySeconds: 5
```

```
periodSeconds: 10

# 3. Exec Command – success if exit code 0
livenessProbe:
exec:
command:
- /bin/sh
- -c
- redis-cli ping | grep PONG
periodSeconds: 5

# Startup probe – for slow-starting containers
# Gives app up to 5 * 60 = 300 seconds to start
startupProbe:
httpGet:
path: /actuator/health
port: 8080
failureThreshold: 60
periodSeconds: 5
```

CHAPTER 3

ReplicaSets – Self-Healing Pod Management

Ensuring Desired Pod Count, Label Selectors & Scaling

3.1 What is a ReplicaSet?

ReplicaSet

A ReplicaSet ensures that a specified number (replicas) of pod copies are running at all times. If a pod crashes or a node goes down, the ReplicaSet controller automatically creates a new pod to replace it. If there are too many pods, it terminates the excess. ReplicaSets use label selectors to identify the pods they manage.

- When a pod dies, the ReplicaSet controller detects the count dropped below desired replicas.
- It immediately creates a new pod on an available node.
- The new pod gets the same labels and spec but a new name and IP.
- ReplicaSets do NOT manage rolling updates — Deployments do.
- You almost never create ReplicaSets directly; Deployments create them for you.

3.2 ReplicaSet YAML

ReplicaSet YAML with selector and pod template

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
name: order-service-rs
namespace: production
labels:
app: order-service
spec:
replicas: 3 # desired number of pod replicas
selector: # how the RS identifies "its" pods
matchLabels:
app: order-service
version: "2.1"
template: # pod template – same as a Pod spec
metadata:
labels: # MUST match selector above
app: order-service
```

```
version: "2.1"
spec:
containers:
- name: order-service
image: myregistry/order-service:v2.1
ports:
- containerPort: 8080
resources:
requests:
memory: "256Mi"
cpu: "250m"
limits:
memory: "512Mi"
cpu: "500m"
readinessProbe:
httpGet:
path: /actuator/health/readiness
port: 8080
initialDelaySeconds: 20
periodSeconds: 5
```

ReplicaSet commands and self-healing demo

```
# ■■ ReplicaSet Commands ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl apply -f replicaset.yaml
kubectl get rs # list ReplicaSets
kubectl get rs order-service-rs -o wide # extra info
kubectl describe rs order-service-rs # detailed info + events

# Scale up/down
kubectl scale rs order-service-rs --replicas=5
kubectl scale rs order-service-rs --replicas=1

# Delete RS (deletes pods too)
kubectl delete rs order-service-rs
# Delete RS but keep pods (orphan them)
kubectl delete rs order-service-rs --cascade=orphan

# Watch pods being created/deleted
kubectl get pods -l app=order-service -w

# Test self-healing: delete a pod and watch RS recreate it
kubectl delete pod order-service-rs-abc12
kubectl get pods -l app=order-service # new pod appears immediately
```

CHAPTER 4

Deployments – Declarative Updates & Rollbacks

Rolling Updates, Rollback, Deployment Strategy & Complete Reference

4.1 What is a Deployment?

Deployment

A Deployment is a higher-level abstraction that manages ReplicaSets. While a ReplicaSet ensures your pods are running, a Deployment adds declarative update management: rolling out a new version, pausing a rollout, rolling back to a previous version, and scaling. A Deployment owns a ReplicaSet, which owns Pods. This is the standard way to run stateless applications in Kubernetes.

Deployment manages ReplicaSets hierarchy

```
Deployment
|-- manages --> ReplicaSet (v2.1) [ACTIVE: 3 replicas]
| |-- Pod (order-service-7d9b5-abc)
| |-- Pod (order-service-7d9b5-def)
| |-- Pod (order-service-7d9b5-ghi)
|
|-- retains --> ReplicaSet (v2.0) [INACTIVE: 0 replicas, kept for rollback]

# When you update the image, Deployment creates a NEW ReplicaSet
# and gradually shifts replicas from old RS to new RS (rolling update)
```

4.2 Complete Deployment YAML

Complete Deployment YAML with all fields

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: order-service
namespace: production
labels:
app: order-service
annotations:
kubernetes.io/change-cause: "v2.1 - Added payment gateway"
spec:
replicas: 3

selector:
matchLabels:
```

```
  app: order-service

  # ■■ Update Strategy ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1 # max pods ABOVE desired count during update
      maxUnavailable: 0 # zero pods unavailable (zero-downtime)

  # ■■ Keep history for rollback ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  revisionHistoryLimit: 5 # keep last 5 RS revisions

  # ■■ Minimum ready time ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  minReadySeconds: 10 # pod must be Ready for 10s before counted as available

  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
        - name: order-service
          image: myregistry/order-service:v2.1
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
              name: http
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: kubernetes
            - name: DB_URL
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: url
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "1000m"
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8080
            initialDelaySeconds: 20
            periodSeconds: 5
```

```
livenessProbe:
httpGet:
path: /actuator/health/liveness
port: 8080
initialDelaySeconds: 30
periodSeconds: 10
terminationGracePeriodSeconds: 60
imagePullSecrets:
- name: registry-credentials
```

4.3 Rolling Updates & Rollbacks

Rolling updates, rollbacks and deployment commands

```
# ■■ Update image (triggers rolling update) ■■■■■■■■■■■■■■■■■■■■
kubectl set image deployment/order-service \
order-service=myregistry/order-service:v2.2

# Or edit the YAML and apply:
kubectl apply -f deployment.yaml

# ■■ Monitor rollout ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl rollout status deployment/order-service
# Waiting for rollout to finish: 1 out of 3 new replicas updated...
# Waiting for rollout to finish: 2 out of 3 new replicas updated...
# deployment "order-service" successfully rolled out

# ■■ View rollout history ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl rollout history deployment/order-service
# REVISION CHANGE-CAUSE
# 1 v2.0 - Initial release
# 2 v2.1 - Added payment gateway
# 3 v2.2 - Performance improvements

kubectl rollout history deployment/order-service --revision=2

# ■■ Rollback ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl rollout undo deployment/order-service # to previous version
kubectl rollout undo deployment/order-service --to-revision=1

# ■■ Pause & Resume (controlled rollout) ■■■■■■■■■■■■■■■■■■■■■■
kubectl rollout pause deployment/order-service
# Make multiple changes...
kubectl set image deployment/order-service order-service=myregistry/order-service:v2.3
kubectl set resources deployment/order-service -c order-service --limits=memory=1Gi
kubectl rollout resume deployment/order-service # roll out all changes at once

# ■■ Deployment Commands ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl get deployments
kubectl get deploy -n production
kubectl describe deployment order-service
kubectl scale deployment order-service --replicas=5
```

```
kubectl delete deployment order-service
```

CHAPTER 5

Services – Stable Network Access to Pods

ClusterIP, NodePort, LoadBalancer, DNS & Service Discovery

5.1 Why Services? The Pod IP Problem

Service

Pods are ephemeral. When a pod dies and is recreated, it gets a NEW IP address. A Service provides a stable, permanent IP address and DNS name that clients use to reach pods. The Service uses label selectors to find pods, load-balances traffic across them, and updates its endpoint list automatically as pods come and go.

Service Type	Accessible From	Use Case	How
ClusterIP	Inside cluster only	Service-to-service communication	Virtual IP on cluster network
NodePort	Outside cluster via node	Dev/testing, on-prem without LB	Port 30000-32767 on every node
LoadBalancer	Outside cluster via LB	Production on cloud providers	Cloud LB (AWS ALB, GCP LB)
ExternalName	Inside cluster only	Alias to external DNS name	DNS CNAME record
Headless	Inside cluster (DNS)	StatefulSets, direct pod access	No ClusterIP; returns pod IPs

5.2 ClusterIP Service – Internal Communication

ClusterIP Service YAML and DNS

```
# ClusterIP – default type; accessible only inside the cluster
apiVersion: v1
kind: Service
metadata:
name: order-service
namespace: production
labels:
app: order-service
spec:
type: ClusterIP # default – can omit this line
selector: # routes to pods with these labels
app: order-service
ports:
- name: http
```

```
  protocol: TCP
  port: 80 # port the Service listens on
  targetPort: 8080 # port the pod/container listens on

  # Optional: pin ClusterIP (usually auto-assigned)
  # clusterIP: 10.96.100.200

  # After applying:
  # - Service gets a stable ClusterIP: 10.96.100.200
  # - DNS: order-service.production.svc.cluster.local
  # - Any pod in cluster can call: http://order-service:80
  # - Or full DNS: http://order-service.production.svc.cluster.local:80
```

5.3 NodePort Service – External Access

NodePort Service YAML

```
  # NodePort – opens a port on EVERY node in the cluster
  apiVersion: v1
  kind: Service
  metadata:
  name: order-service-nodeport
  spec:
  type: NodePort

  selector:
  app: order-service

  ports:
  - protocol: TCP
  port: 80 # ClusterIP port (internal)
  targetPort: 8080 # Pod port
  nodePort: 30080 # Port on each node (30000-32767)
  # Omit to get auto-assigned port

  # After applying:
  # Access via: http://:30080
  # On minikube: minikube service order-service-nodeport --url
```

5.4 LoadBalancer Service – Cloud Load Balancer

LoadBalancer Service YAML with cloud annotations

```
  # LoadBalancer – provisions cloud LB (AWS ALB, GCP LB, Azure LB)
  apiVersion: v1
  kind: Service
  metadata:
  name: order-service-lb
  annotations:
  # AWS-specific annotations
  service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
  service.beta.kubernetes.io/aws-load-balancer-cross-zone-load-balancing-enabled: "true"
```

```
spec:
type: LoadBalancer

selector:
app: order-service

ports:
- name: http
protocol: TCP
port: 80
targetPort: 8080
- name: https
protocol: TCP
port: 443
targetPort: 8443

# After applying:
kubectl get svc order-service-lb
# NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
# order-service-lb LoadBalancer 10.96.200.5 a1b2c3.aws.com 80:31234/TCP

# Access via: http://a1b2c3.aws.com
```

5.5 Service DNS & Discovery

DNS patterns, environment variables, service commands

```
# Kubernetes DNS pattern for Services:
# ..svc.
# Default cluster domain: cluster.local

# Examples:
# order-service.production.svc.cluster.local (full DNS)
# order-service.production (cross-namespace short)
# order-service (same namespace short)

# Environment variables K8s injects into pods:
# ORDER_SERVICE_SERVICE_HOST = 10.96.100.200
# ORDER_SERVICE_SERVICE_PORT = 80

# DNS lookup inside a pod:
kubectl exec -it some-pod -- nslookup order-service
# Server: 10.96.0.10 (CoreDNS)
# Address: 10.96.0.10
# Name: order-service.production.svc.cluster.local
# Address: 10.96.100.200

# ■■ Service Commands ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl get services
kubectl get svc -n production
kubectl describe svc order-service
kubectl get endpoints order-service # see actual pod IPs behind service
kubectl delete svc order-service
```

CHAPTER 6

ConfigMaps & Secrets

Externalised Configuration, Sensitive Data & Environment Injection

6.1 ConfigMaps

ConfigMap

A ConfigMap stores non-sensitive configuration data as key-value pairs or files. It decouples environment-specific configuration from container images, allowing the same image to run in dev, staging, and prod with different configurations. ConfigMaps can be consumed as environment variables, command-line arguments, or mounted as files inside containers.

ConfigMap creation and consumption patterns

```
# ■■ Create ConfigMap from literals ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl create configmap app-config \
--from-literal=app.name=OrderService \
--from-literal=app.version=2.1 \
--from-literal=log.level=INFO

# ■■ Create ConfigMap from file ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl create configmap app-config --from-file=application.properties
kubectl create configmap nginx-config --from-file=nginx.conf

# ■■ ConfigMap YAML ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
apiVersion: v1
kind: ConfigMap
metadata:
name: app-config
namespace: production
data:
# Key-value pairs
APP_NAME: "OrderService"
APP_VERSION: "2.1"
LOG_LEVEL: "INFO"
DB_HOST: "postgres-service"
DB_PORT: "5432"

# Multi-line config file
application.properties: |
spring.application.name=order-service
spring.profiles.active=kubernetes
server.port=8080
spring.datasource.url=jdbc:postgresql://postgres-service:5432/orderdb
```

```
logging.level.com.example=INFO

# ■■ Use ConfigMap as environment variables ■■■■■■■■■■■■■■■■■■■■■
spec:
containers:
- name: app
image: myapp:v2
# Inject all ConfigMap keys as env vars
envFrom:
- configMapRef:
name: app-config
# Or inject specific keys
env:
- name: LOG_LEVEL
valueFrom:
configMapKeyRef:
name: app-config
key: LOG_LEVEL

# ■■ Mount ConfigMap as files ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
volumeMounts:
- name: config-volume
mountPath: /app/config # creates /app/config/application.properties
volumes:
- name: config-volume
configMap:
name: app-config
items:
- key: application.properties
path: application.properties
```

6.2 Secrets

Secret

A Secret stores sensitive data such as passwords, tokens, and TLS certificates. Secret values are base64-encoded (NOT encrypted by default — enable encryption at rest in production). Secrets work the same way as ConfigMaps in terms of consumption but should be handled with extra care around RBAC and access controls.

Secret Type	Use For
Opaque (default)	Arbitrary key-value data: passwords, API keys, connection strings
kubernetes.io/tls	TLS certificate and key (cert and key fields required)
kubernetes.io/dockerconfigjson	Docker registry credentials for pulling images
kubernetes.io/service-account-token	Service account auth token (auto-created)
kubernetes.io/ssh-auth	SSH private key

Secret Type	Use For
kubernetes.io/basic-auth	Basic auth credentials (username + password)

Secret creation and all consumption patterns

```
# ■■ Create Secret from literals ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl create secret generic db-secret \
--from-literal=username=orderuser \
--from-literal=password=SuperSecret123!

# ■■ Create TLS Secret ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl create secret tls tls-secret \
--cert=tls.crt --key=tls.key

# ■■ Create Docker registry secret ■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl create secret docker-registry registry-secret \
--docker-server=registry.example.com \
--docker-username=myuser \
--docker-password=mypassword

# ■■ Secret YAML ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
apiVersion: v1
kind: Secret
metadata:
name: db-secret
namespace: production
type: Opaque
data:
# Values MUST be base64-encoded
# echo -n "orderuser" | base64 -> b3JkZXJ1c2Vy
username: b3JkZXJ1c2Vy
password: U3VwZXJTZWNyZXQxMjMh

# ■■ Use Secret as env vars ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
spec:
containers:
- name: app
env:
- name: DB_USERNAME
valueFrom:
secretKeyRef:
name: db-secret
key: username
- name: DB_PASSWORD
valueFrom:
secretKeyRef:
name: db-secret
key: password
# Or inject ALL secret keys as env vars:
```

```
envFrom:
- secretRef:
name: db-secret

# ■■ Mount Secret as files ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
volumeMounts:
- name: secret-vol
mountPath: /app/secrets
readOnly: true
volumes:
- name: secret-vol
secret:
secretName: db-secret
defaultMode: 0400 # read-only by owner only
```

CHAPTER 7

Namespaces, Labels & Selectors

Resource Isolation, Quotas, Metadata & Filtering

7.1 Namespaces – Virtual Clusters

Namespace

Namespaces provide a mechanism for isolating groups of resources within a single cluster. They are useful for separating environments (dev/staging/prod), teams, or projects in a shared cluster. Resource names must be unique within a namespace but can repeat across namespaces.

Default Namespace	Purpose
default	Resources with no explicit namespace go here
kube-system	Kubernetes system components (CoreDNS, kube-proxy, etc.)
kube-public	Publicly accessible resources (cluster info)
kube-node-lease	Node heartbeat lease objects

Namespaces, ResourceQuota and LimitRange

```
# ■■ Create namespace ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl create namespace production
kubectl create namespace staging
kubectl create namespace monitoring
# ■■ Namespace YAML ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
apiVersion: v1
kind: Namespace
metadata:
name: production
labels:
env: production
team: backend
# ■■ ResourceQuota – limit resource usage per namespace ■■■■■■■■
apiVersion: v1
kind: ResourceQuota
metadata:
name: production-quota
namespace: production
spec:
hard:
```

```
  pods: "20"
  requests.cpu: "10"
  requests.memory: 20Gi
  limits.cpu: "20"
  limits.memory: 40Gi
  services: "10"
  persistentvolumeclaims: "5"

# ■■ LimitRange – default limits for pods ■■■■■■■■■■■■■■■■■■■■■■
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: production
spec:
  limits:
  - type: Container
    default:
      cpu: "500m"
      memory: 512Mi
    defaultRequest:
      cpu: "250m"
      memory: 256Mi
    max:
      cpu: "2"
      memory: 2Gi
    min:
      cpu: "100m"
      memory: 64Mi
```

7.2 Labels & Selectors

Labels, selectors and annotations

```
# ■■ Labels on resources ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Add/update labels
kubectl label pod my-pod env=production
kubectl label pod my-pod version=v2.1 --overwrite

# Remove label
kubectl label pod my-pod version-

# ■■ Filtering with label selectors ■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl get pods -l app=order-service
kubectl get pods -l app=order-service,env=production
kubectl get pods -l "env in (production,staging)"
kubectl get pods -l "version notin (v1.0,v1.1)"
kubectl get pods -l "!deprecated" # pods WITHOUT deprecated label
kubectl get all -l app=order-service # all object types
```

```
# ■■ Selector in YAML (Deployment/Service) ■■■■■■■■■■■■■■■■■■■■■■
selector:
matchLabels:
app: order-service
env: production

# matchExpressions (more powerful)
selector:
matchExpressions:
- key: app
operator: In
values: [order-service, payment-service]
- key: env
operator: NotIn
values: [dev]
- key: tier
operator: Exists

# ■■ Annotations (non-identifying metadata) ■■■■■■■■■■■■■■■■■■■■
metadata:
annotations:
description: "Main order processing API"
contact: "team@example.com"
prometheus.io/scrape: "true"
prometheus.io/path: "/actuator/prometheus"
prometheus.io/port: "8080"
kubernetes.io/change-cause: "v2.1 feature update"
```

CHAPTER 8

Ingress – HTTP/HTTPS Routing into the Cluster

Ingress Controller, Path Routing, Host Routing & TLS Termination

8.1 What is Ingress?

Ingress

An Ingress is a Kubernetes API object that manages external access to services, typically HTTP/HTTPS. It provides URL path-based routing, hostname-based virtual hosting, TLS termination, and load balancing — all with a single external IP or load balancer, routing to multiple backend services. An Ingress Controller (e.g. nginx-ingress, Traefik, AWS ALB Controller) must be running for Ingress resources to work.

Ingress with path routing, host routing and TLS

```
# Install NGINX Ingress Controller (most common)
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/cloud/deploy.yaml

# On minikube:
minikube addons enable ingress

# ■■ Ingress YAML – Path-based routing ■■■■■■■■■■■■■■■■■■■■■■■■
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecommerce-ingress
  namespace: production
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "60"
    nginx.ingress.kubernetes.io/rate-limit: "100"
spec:
  ingressClassName: nginx

  # ■■ TLS termination ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  tls:
    - hosts:
        - api.myshop.com
      secretName: tls-secret # kubectl create secret tls tls-secret ...

  rules:
    # ■■ Host-based routing ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
- host: api.myshop.com
http:
paths:
# Path-based routing: /api/orders -> order-service
- path: /api/orders
pathType: Prefix
backend:
service:
name: order-service
port:
number: 80

- path: /api/products
pathType: Prefix
backend:
service:
name: product-service
port:
number: 80

- path: /api/payments
pathType: Prefix
backend:
service:
name: payment-service
port:
number: 80

- path: / # catch-all: frontend
pathType: Prefix
backend:
service:
name: frontend-service
port:
number: 80

- host: admin.myshop.com
http:
paths:
- path: /
pathType: Prefix
backend:
service:
name: admin-service
port:
number: 80

# ■■ Commands ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl get ingress
```

```
kubectl describe ingress ecommerce-ingress
```

CHAPTER 9

Storage – PersistentVolumes, Claims & StatefulSets

PV, PVC, StorageClass, Dynamic Provisioning & StatefulSets

9.1 PersistentVolume & PersistentVolumeClaim

Object	Who Creates It	Role
PersistentVolume (PV)	Admin (or dynamically by StorageClass)	Represents actual storage (NFS, EBS, GCE disk)
PersistentVolumeClaim (PVC)	Developer / Pod spec	Request for storage: size, access mode, storageClass
StorageClass	Admin	Defines storage type; enables dynamic provisioning

StorageClass, PVC and using PVC in pods

```
# ■■ StorageClass (dynamic provisioning) ■■■■■■■■■■■■■■■■■■■■■■
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: fast-ssd
provisioner: ebs.csi.aws.com # AWS EBS CSI driver
parameters:
type: gp3
iops: "3000"
throughput: "125"
encrypted: "true"
reclaimPolicy: Retain # Retain | Delete | Recycle
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true

# ■■ PersistentVolumeClaim ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: postgres-pvc
namespace: production
spec:
storageClassName: fast-ssd # matches StorageClass name
accessModes:
```

```
- ReadWriteOnce # RWO | ROX | RWX | RWOP
resources:
requests:
storage: 20Gi

# ■■ Use PVC in a Pod ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
spec:
containers:
- name: postgres
image: postgres:16
volumeMounts:
- name: postgres-storage
mountPath: /var/lib/postgresql/data
volumes:
- name: postgres-storage
persistentVolumeClaim:
claimName: postgres-pvc
```

9.2 StatefulSet – Stateful Applications

StatefulSet

StatefulSet is like a Deployment but for stateful applications (databases, Kafka, Elasticsearch). It gives each pod a stable, unique identity (pod-0, pod-1, pod-2), stable persistent storage that follows the pod even after rescheduling, and ordered startup/shutdown (pod-0 starts first, pod-2 shuts down first).

StatefulSet for database with volumeClaimTemplates

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
name: postgres
namespace: production
spec:
serviceName: postgres-headless # headless service for DNS
replicas: 3
selector:
matchLabels:
app: postgres
template:
metadata:
labels:
app: postgres
spec:
containers:
- name: postgres
image: postgres:16
ports:
- containerPort: 5432
```

```
name: postgres
env:
- name: POSTGRES_PASSWORD
valueFrom:
secretKeyRef:
name: postgres-secret
key: password
volumeMounts:
- name: postgres-data
mountPath: /var/lib/postgresql/data

# Each pod gets its own PVC automatically
volumeClaimTemplates:
- metadata:
name: postgres-data
spec:
storageClassName: fast-ssd
accessModes: [ReadWriteOnce]
resources:
requests:
storage: 50Gi

# Pods created: postgres-0, postgres-1, postgres-2
# DNS: postgres-0.postgres-headless.production.svc.cluster.local
# Each gets its own PVC: postgres-data-postgres-0, postgres-data-postgres-1
```

CHAPTER 10

Autoscaling & Microservices on Kubernetes

HPA, VPA, Full E-Commerce Deployment & Best Practices

10.1 Horizontal Pod Autoscaler (HPA)

HPA

The Horizontal Pod Autoscaler automatically scales the number of pod replicas in a Deployment, ReplicaSet, or StatefulSet based on observed CPU utilisation, memory utilisation, or custom metrics. The HPA controller checks metrics every 15 seconds and adjusts the replica count to meet the target utilisation.

HPA with CPU, memory and custom metrics

```
# ■■ HPA YAML ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: order-service-hpa
namespace: production
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: order-service

minReplicas: 2 # minimum replicas even at 0% load
maxReplicas: 20 # maximum replicas

metrics:
# CPU-based scaling
- type: Resource
resource:
name: cpu
target:
type: Utilization
averageUtilization: 70 # scale up when avg CPU > 70%

# Memory-based scaling
- type: Resource
resource:
name: memory
target:
```

```
  type: AverageValue
  averageValue: 400Mi

  # Custom metric (Prometheus/KEDA)
  - type: External
  external:
  metric:
  name: http_requests_per_second
  selector:
  matchLabels:
  service: order-service
  target:
  type: AverageValue
  averageValue: "100"

  behavior:
  scaleDown:
  stabilizationWindowSeconds: 300 # wait 5 min before scaling down
  policies:
  - type: Pods
  value: 2
  periodSeconds: 60 # remove at most 2 pods per minute
  scaleUp:
  stabilizationWindowSeconds: 0 # scale up immediately
  policies:
  - type: Pods
  value: 4
  periodSeconds: 15 # add up to 4 pods per 15 seconds

  # Commands
  kubectl get hpa
  kubectl describe hpa order-service-hpa
  # Shows: current replicas, targets, min/max, recent events
```

10.2 Microservices Architecture on Kubernetes

Microservices on Kubernetes

Kubernetes is the natural platform for running microservices. Each microservice runs as an independent Deployment with its own Pods, Service, ConfigMap, and Secrets. Services communicate via ClusterIP services using DNS names. An Ingress routes external traffic. Each service can be scaled, updated, and rolled back independently.

E-Commerce microservices directory structure

```
  # E-Commerce Platform – Full Kubernetes Structure

  ecommerce-platform/
  |-- namespaces/
  | |-- production.yaml
  | |-- monitoring.yaml
```

```
|
|-- infrastructure/
| |-- postgres-statefulset.yaml # Database
| |-- postgres-service.yaml # ClusterIP
| |-- redis-deployment.yaml # Cache
| |-- redis-service.yaml
| |-- kafka-statefulset.yaml # Message broker
|
|-- services/
| |-- user-service/
| | |-- deployment.yaml
| | |-- service.yaml # ClusterIP
| | |-- configmap.yaml
| | |-- hpa.yaml
| |-- product-service/
| | |-- deployment.yaml
| | |-- service.yaml
| | |-- configmap.yaml
| | |-- hpa.yaml
| |-- order-service/
| | |-- deployment.yaml
| | |-- service.yaml
| | |-- configmap.yaml
| | |-- hpa.yaml
| |-- payment-service/
| | |-- deployment.yaml
| | |-- service.yaml
| |-- notification-service/
| |-- deployment.yaml
| |-- service.yaml
|
|-- secrets/
| |-- db-secret.yaml
| |-- registry-secret.yaml
|
|-- ingress/
|-- ingress.yaml # Routes /api/* to services
```

Deploying the microservices stack

```
# ■■ Deploy everything with kubectl ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kubectl apply -f namespaces/
kubectl apply -f infrastructure/
kubectl apply -f secrets/
kubectl apply -f services/
kubectl apply -f ingress/

# ■■ Or use Kustomize ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: production
resources:
- namespaces/
- infrastructure/
- services/
- ingress/

kubectl apply -k .

# ■■ Service communication example ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Order service calls User service by DNS name:
# http://user-service/api/users/{id}

# Order service calls Product service:
# http://product-service/api/products/{id}

# All services communicate via ClusterIP – no external exposure needed
```

10.3 Kubernetes Best Practices

Area	Best Practice
Resources	Always set requests and limits for every container. No limits = resource starvation.
Probes	Always add readinessProbe. Add livenessProbe only when you know what to check.
Images	Pin image tags to specific versions (not 'latest'). Use imagePullPolicy: Always.
Security	Run as non-root user. Use read-only root filesystem. Drop capabilities.
Namespaces	Separate environments (dev/staging/prod) into different namespaces.
Labels	Label everything consistently: app, version, env, tier, team.
ConfigMaps	Externalise all environment-specific config into ConfigMaps, not images.
Secrets	Never store secrets in code or ConfigMaps. Use Secrets + RBAC + Vault.
Rolling Updates	Set maxUnavailable=0 and minReadySeconds=10 for zero-downtime deployments.
Health Checks	Use /actuator/health/liveness and /actuator/health/readiness in Spring Boot.
HPA	Set minReplicas=2 (never 1) for high availability. Define scale-down policies.
Networking	Use NetworkPolicy to restrict pod-to-pod communication to only what's needed.
Namespace Quotas	Set ResourceQuota on every namespace to prevent runaway resource usage.
Monitoring	Annotate pods for Prometheus scraping. Alert on pod crashes and high lag.
RBAC	Use least-privilege ServiceAccounts. Never use default ServiceAccount.
Graceful Shutdown	Handle SIGTERM in your app. Set terminationGracePeriodSeconds appropriately.

+ The most important Kubernetes habit: always set resource requests and limits, add a readinessProbe, and never use 'latest' as your image tag. These three practices prevent 80% of production incidents.

! Pods without resource limits can consume all CPU and memory on a node, crashing other pods. Always set limits, and set LimitRange defaults at the namespace level to enforce this automatically.

Quick Reference – kubectl Commands

Command	Purpose
kubectl get pods / -o wide / -A	List pods (current ns / with node info / all namespaces)
kubectl get all -n production	List all resources in a namespace
kubectl describe pod my-pod	Detailed pod info + recent events
kubectl logs -f pod/my-pod	Follow pod logs
kubectl exec -it my-pod -- bash	Interactive shell in pod
kubectl apply -f manifest.yaml	Create/update resources from YAML
kubectl delete -f manifest.yaml	Delete resources defined in YAML
kubectl get deploy / rs / svc / ing	List deployments / replicasets / services / ingresses
kubectl scale deploy/app --replicas=5	Scale a deployment to 5 replicas
kubectl set image deploy/app app=img:v2	Update deployment image (rolling update)
kubectl rollout status deploy/app	Watch rolling update progress
kubectl rollout history deploy/app	Show deployment revision history
kubectl rollout undo deploy/app	Rollback to previous version
kubectl port-forward svc/app 8080:80	Forward local port to service (dev access)
kubectl get configmap / secret	List ConfigMaps / Secrets
kubectl create secret generic name --from-literal=k=v	Create a secret
kubectl get pvc / pv	List PersistentVolumeClaims / PersistentVolumes
kubectl get hpa	List HorizontalPodAutoscalers and current state
kubectl top pods / nodes	Live CPU & memory usage (requires metrics-server)
kubectl get events --sort-by=.lastTimestamp	View cluster events sorted by time
kubectl config get-contexts	List all cluster contexts
kubectl config use-context prod	Switch to a different cluster context
kubectl create ns production	Create a namespace
kubectl label pod my-pod key=value	Add/update label on a pod
kubectl get pods -l app=order-service	Filter pods by label selector